```
# Market regime analysis starter.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# For HMM analysis: install hmmlearn
# Not a Python command.  It installs the library on Colab
!pip install hmmlearn
# https://anaconda.org/conda-forge/hmmlearn
from hmmlearn import hmm
```

```
Collecting hmmlearn
  Downloading hmmlearn-0.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.me
Requirement already satisfied: numpy>=1.10 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: scikit-learn!=0.22.0,>=0.16 in /usr/local/lib/python3.11/di
Requirement already satisfied: scipy>=0.19 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-pack
Downloading hmmlearn-0.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (165
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 165.9/165.9 kB 9.7 MB/s eta 0:00:00
Installing collected packages: hmmlearn
Successfully installed hmmlearn-0.3.3
```

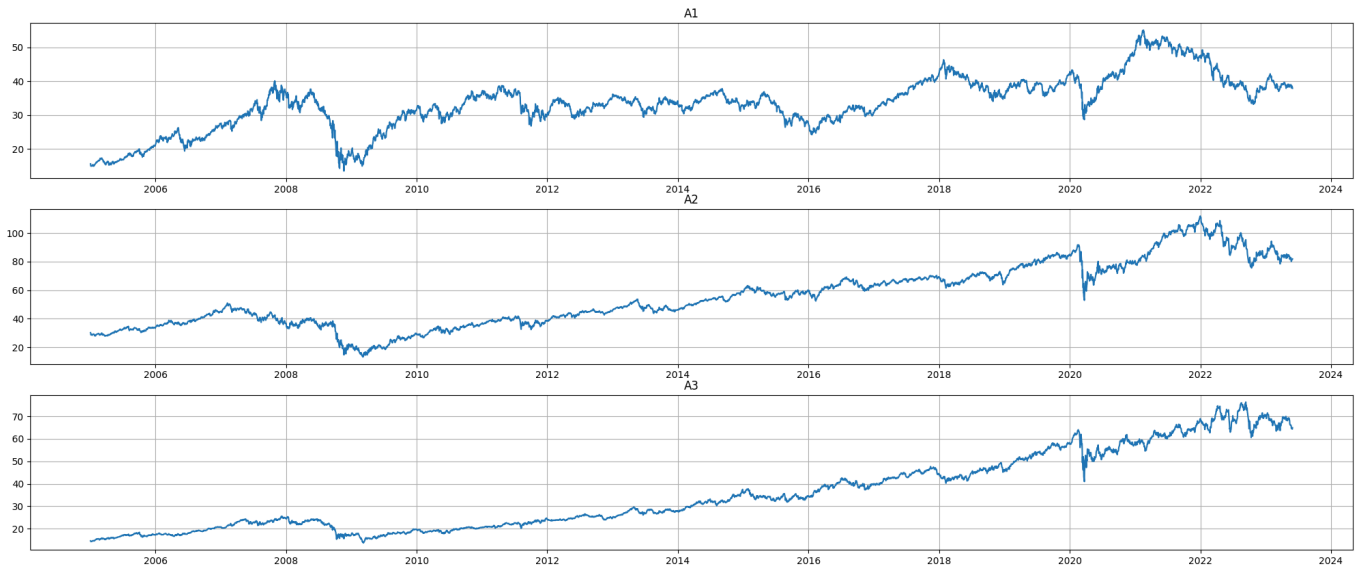
```
# read asset prices dataset from Github into a pandas dataframe
# (MODIFY file name for your group!)
fcsv = "/content/assets_regime_analysis_4.csv"
df = pd.read_csv(fcsv, index_col=[0], parse_dates=['Date'])
df
```

	A1	A2	A3
Date			
2005-01-03	15.559469	30.156376	14.526597
2005-01-04	15.080416	29.742468	14.426337
2005-01-05	14.895803	28.629606	14.199438
2005-01-06	14.885677	28.874975	14.273320
2005-01-07	14.914498	28.872492	14.273320
...	...	...	...
2023-05-24	38.101505	80.131714	65.470001
2023-05-25	38.022125	80.141663	64.589996
2023-05-26	38.696838	81.067291	64.589996
2023-05-30	38.250336	81.415657	64.339996
2023-05-31	37.893135	81.843636	64.930000

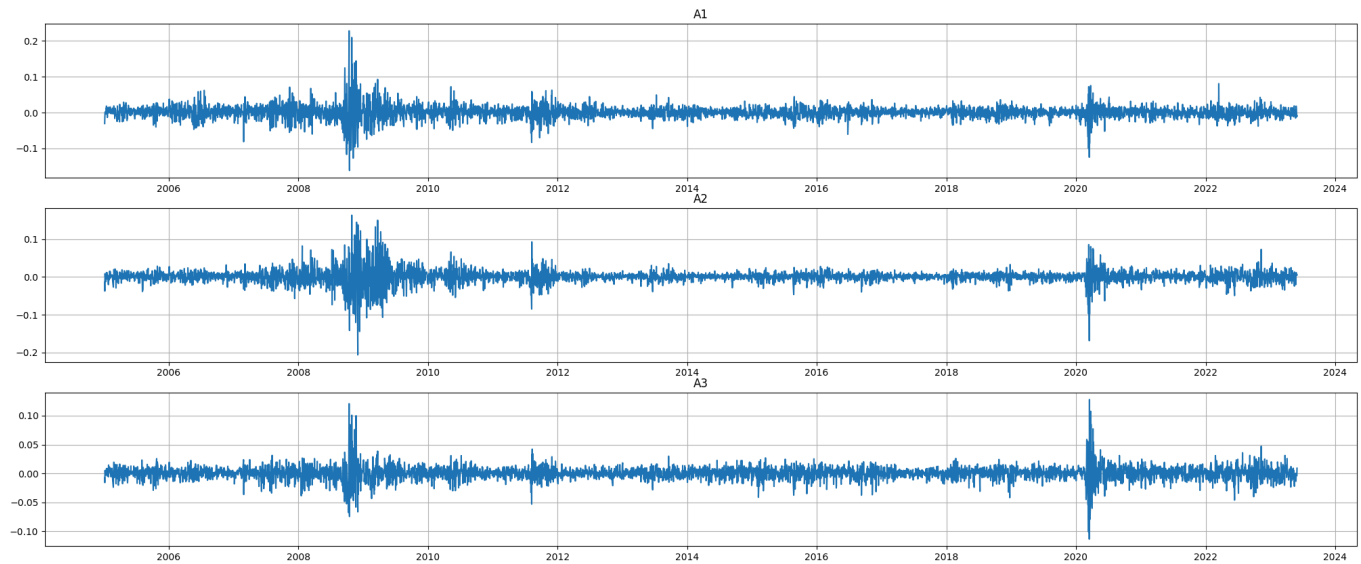
4634 rows × 3 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
tickers = list(df.columns)


# asset prices
plt.figure(figsize = (25, 10))
plt.subplot(3,1,1)
plt.plot(df.index, df[tickers[0]])
plt.title(tickers[0])
plt.grid(True)
plt.subplot(3,1,2)
plt.plot(df.index, df[tickers[1]])
plt.title(tickers[1])
plt.grid(True)
plt.subplot(3,1,3)
plt.plot(df.index, df[tickers[2]])
plt.title(tickers[2])
plt.grid(True)
plt.show()
```




```
# daily returns
plt.figure(figsize = (25, 10))
plt.subplot(3,1,1)
plt.plot(df.index, df[tickers[0]].pct_change())
plt.title(tickers[0])
plt.grid(True)
plt.subplot(3,1,2)
plt.plot(df.index, df[tickers[1]].pct_change())
plt.title(tickers[1])
plt.grid(True)
plt.subplot(3,1,3)
plt.plot(df.index, df[tickers[2]].pct_change())
plt.title(tickers[2])
plt.grid(True)
plt.show()
```




```
def retcolumn(ticker):
    return df[ticker].pct_change().dropna().values
```

```
# excract numerical values only from pandas dataframe (numpy array)
X = np.column_stack((retcolumn(tickers[0]), retcolumn(tickers[1]), retcolumn(tickers[2])))
X
```

```
array([[-0.03078855, -0.01372539, -0.00690181],
       [-0.01224186, -0.03741659, -0.01572812],
       [-0.0006798 ,  0.00857046,  0.00520317],
       ...,
       [ 0.01774528,  0.01154991,  0.        ],
       [-0.01153848,  0.00429724, -0.00387057],
       [-0.0093385 ,  0.00525671,  0.0091701 ]])
```

```
# learning the hidden states (HMM model fit)
model = hmm.GaussianHMM(n_components = 2, covariance_type = "diag", n_iter = 50, random_state
model
```

```
GaussianHMM
GaussianHMM(n_components=2, n_iter=50, random_state=15)
```

```
model.fit(X)
```

```
GaussianHMM
GaussianHMM(n_components=2, n_iter=50, random_state=15)
```

```
# hidden states corresponding to observed X
Z = model.predict(X)
Z
```

```
array([0, 0, 1, ..., 1, 1, 1])
```

```
# distinct states
states = pd.unique(Z)
states
```

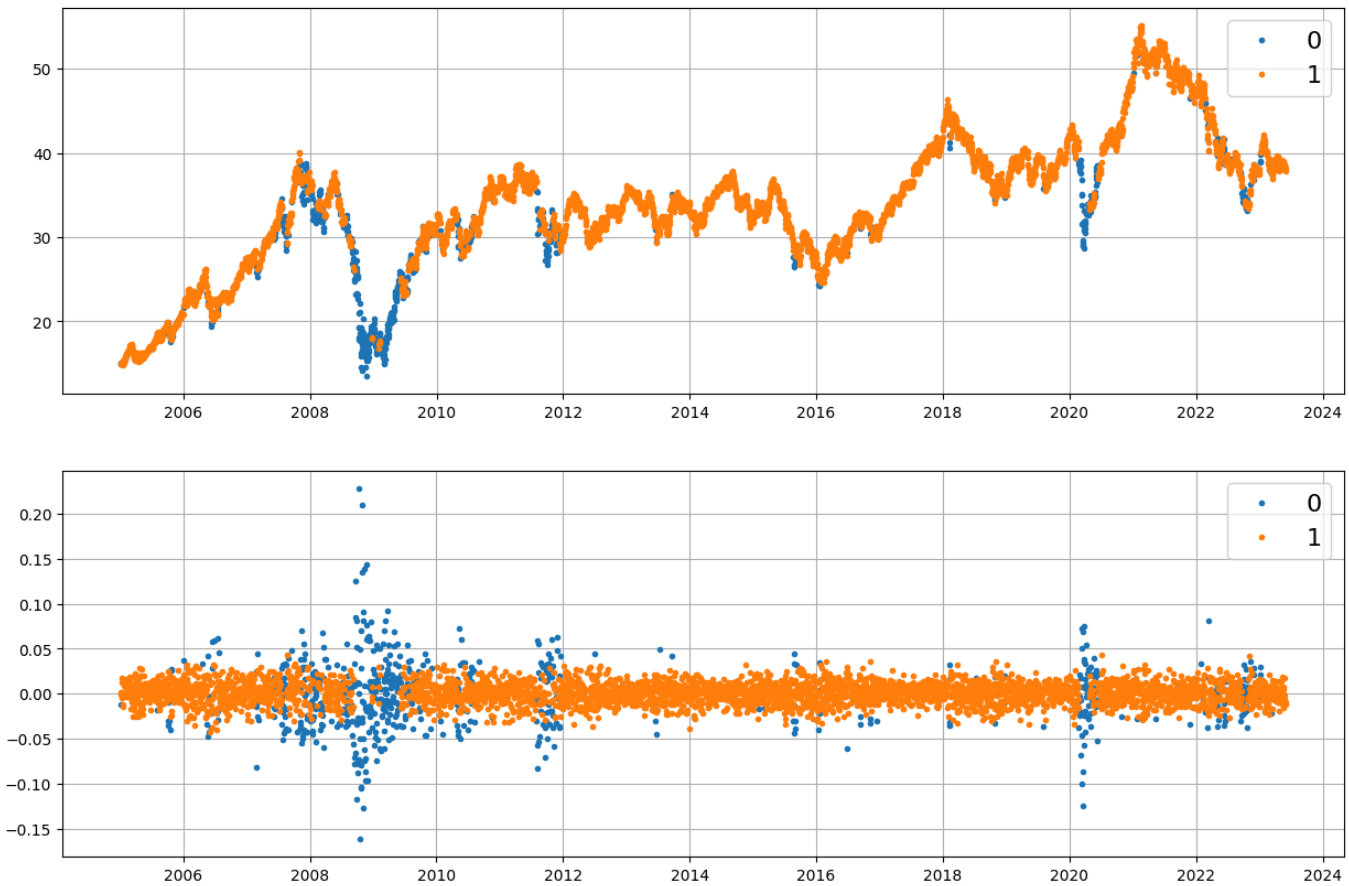
```
array([0, 1])
```

```
# hidden state transition probabilities
model.transmat_
```

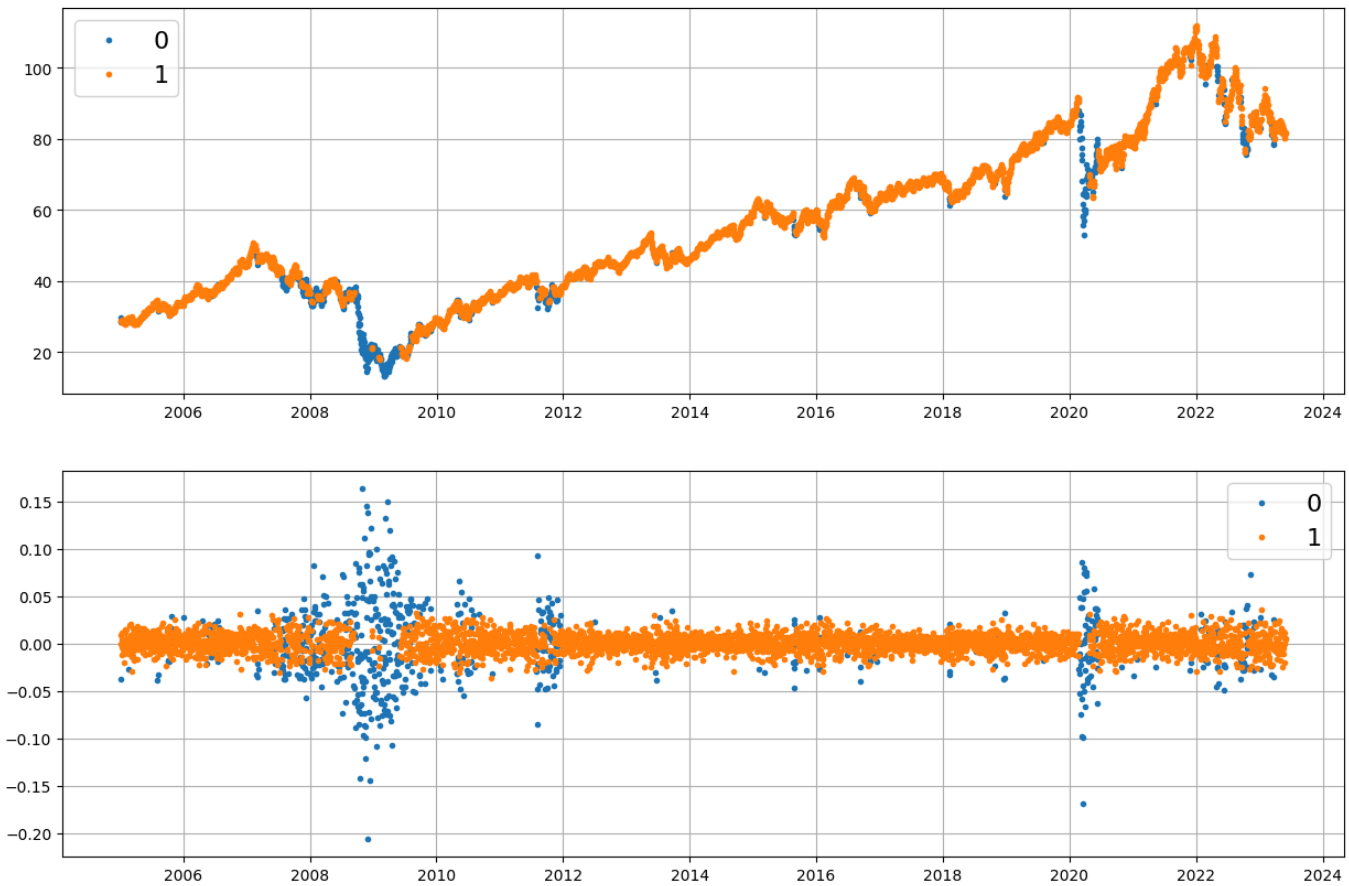
```
array([[0.76311863, 0.23688137],
       [0.04299103, 0.95700897]])
```

```
# regime plotting function (will be called repeatedly for each asset)
def regime_plot(df, nasset):
    plt.figure(figsize = (15, 10))
    plt.subplot(2,1,1)
    for i in states:
        s = (Z == i)
        x = df.index[s]
        y = df[tickers[nasset]].iloc[s]
        plt.plot(x, y, '.')
    plt.legend(states, fontsize=16)
    plt.grid(True)
    plt.subplot(2,1,2)
    for i in states:
        s = (Z == i)
        x = df.index[s]
        y = df[tickers[nasset]].pct_change().iloc[s]
        plt.plot(x, y, '.')
    plt.legend(states, fontsize=16)
    plt.grid(True)
    return plt
```

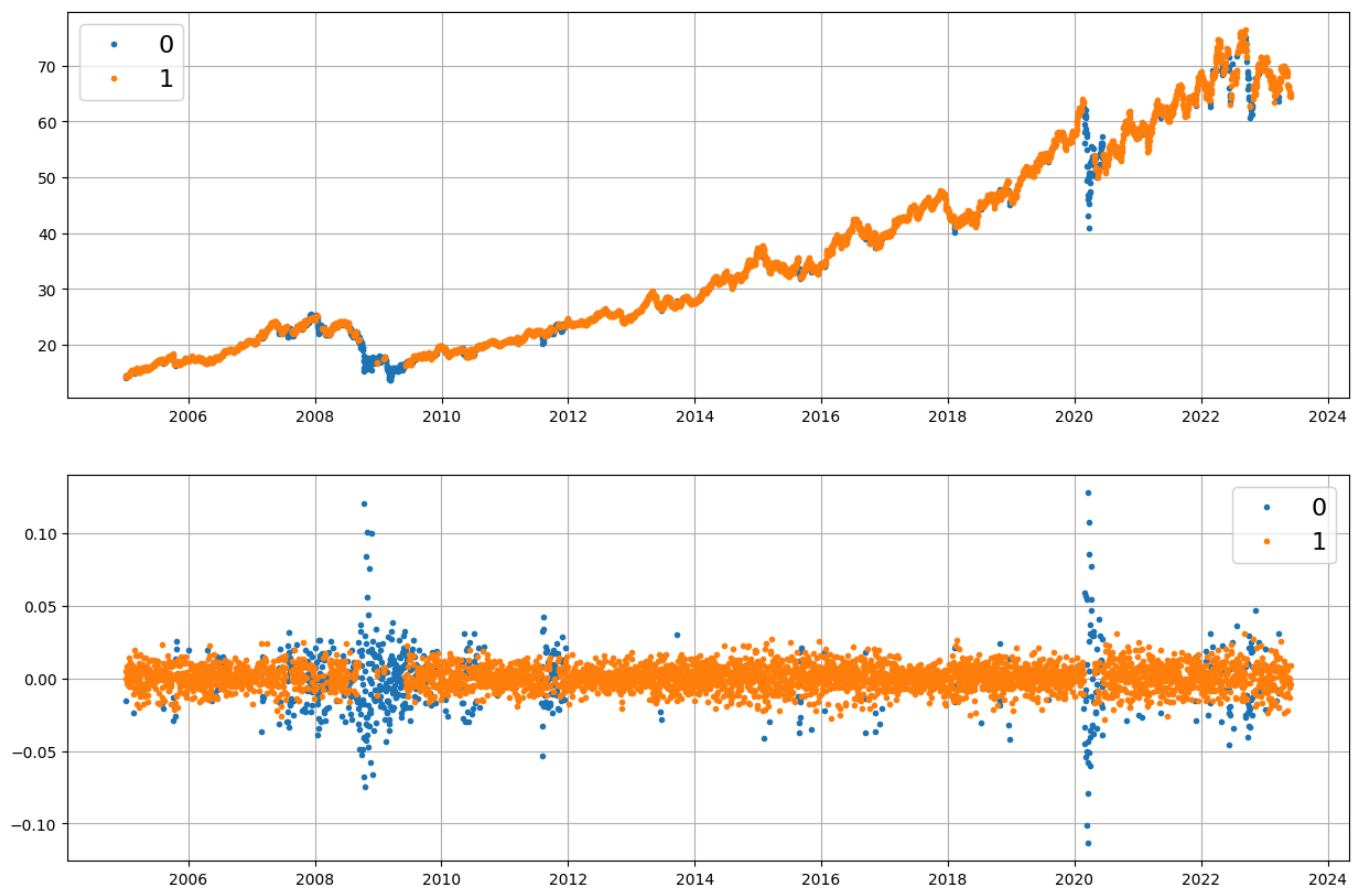
```
# removing the first row of the data that has NA-return
regime_plot(df.iloc[1: , :], 0).show()
```




```
regime_plot(df.iloc[1: , :], 1).show()
```




```
regime_plot(df.iloc[1: , :], 2).show()
```




```
# EXAMPLE: average returns in the first market state (Z == 0)
df.pct_change().dropna().iloc[(Z == 0)].mean()
```

	0
A1	-0.001748
A2	-0.002032
A3	-0.002276

dtype: float64

```
# EXAMPLE: covariance matrix for the second market state (Z == 1)
df.pct_change().dropna().iloc[(Z == 1)].cov()
```

	A1	A2	A3
A1	0.000134	0.000049	0.000026
A2	0.000049	0.000088	0.000040
A3	0.000026	0.000040	0.000064

```
# correlation matrix for the second market state (Z == 1)
df.pct_change().dropna().iloc[(Z == 1)].corr()
```

	A1	A2	A3
A1	1.000000	0.453996	0.280849
A2	0.453996	1.000000	0.530223
A3	0.280849	0.530223	1.000000

PyportfolioOPT frpm the notes

EXTRA WORK below

EXTRA WORK below

EXTRA WORK below

EXTRA WORK below

EXTRA WORK below

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (2.18
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/pytho
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (fro
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/dist-
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (f
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (fr
Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (fr
Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/pyth
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from kera
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from ker
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-
```

```
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (fr
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/pyt
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-pac
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.utils import class_weight
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
```

```
Z = pd.Series(Z, index=returns.index)
data_lstm = returns.copy()
data_lstm['Regime'] = Z
```

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaled = scaler.fit_transform(data_lstm.drop(columns='Regime'))
```

```
X = []
y = []
window_size = 20

for i in range(window_size, len(scaled)):
    X.append(scaled[i - window_size:i])
    y.append(data_lstm['Regime'].iloc[i])

X = np.array(X)
y = np.array(y)
```

```
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

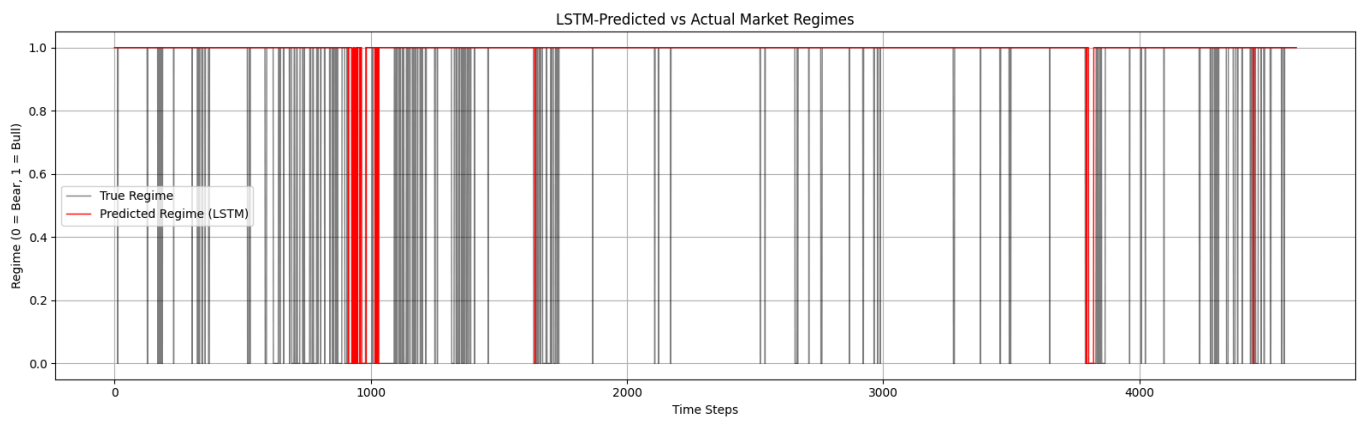
```
cw = class_weight.compute_class_weight(class_weight='balanced', classes=np.unique(y), y=y)
cw = dict(enumerate(cw))

model.fit(X, y, epochs=20, batch_size=32, class_weight=cw)
```

```
predicted = model.predict(X)
predicted_classes = (predicted > 0.5).astype(int).flatten()
```

```
Epoch 1/20
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do r
  super().__init__(**kwargs)
145/145 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.1618 - loss: 0.6974
Epoch 2/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - accuracy: 0.7523 - loss: 0.6849
Epoch 3/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8622 - loss: 0.6809
Epoch 4/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.5973 - loss: 0.6940
Epoch 5/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.7976 - loss: 0.6704
Epoch 6/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.7979 - loss: 0.6721
Epoch 7/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - accuracy: 0.8508 - loss: 0.6783
Epoch 8/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.7654 - loss: 0.6732
Epoch 9/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.7557 - loss: 0.6874
Epoch 10/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8615 - loss: 0.6867
Epoch 11/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8595 - loss: 0.6861
Epoch 12/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8584 - loss: 0.6910
Epoch 13/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 0.5746 - loss: 0.7102
Epoch 14/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.8635 - loss: 0.6883
Epoch 15/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.8560 - loss: 0.6789
Epoch 16/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - accuracy: 0.7332 - loss: 0.6838
Epoch 17/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.7335 - loss: 0.6434
Epoch 18/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8622 - loss: 0.6873
Epoch 19/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - accuracy: 0.8588 - loss: 0.6876
Epoch 20/20
145/145 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.8655 - loss: 0.6887
145/145 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step
```

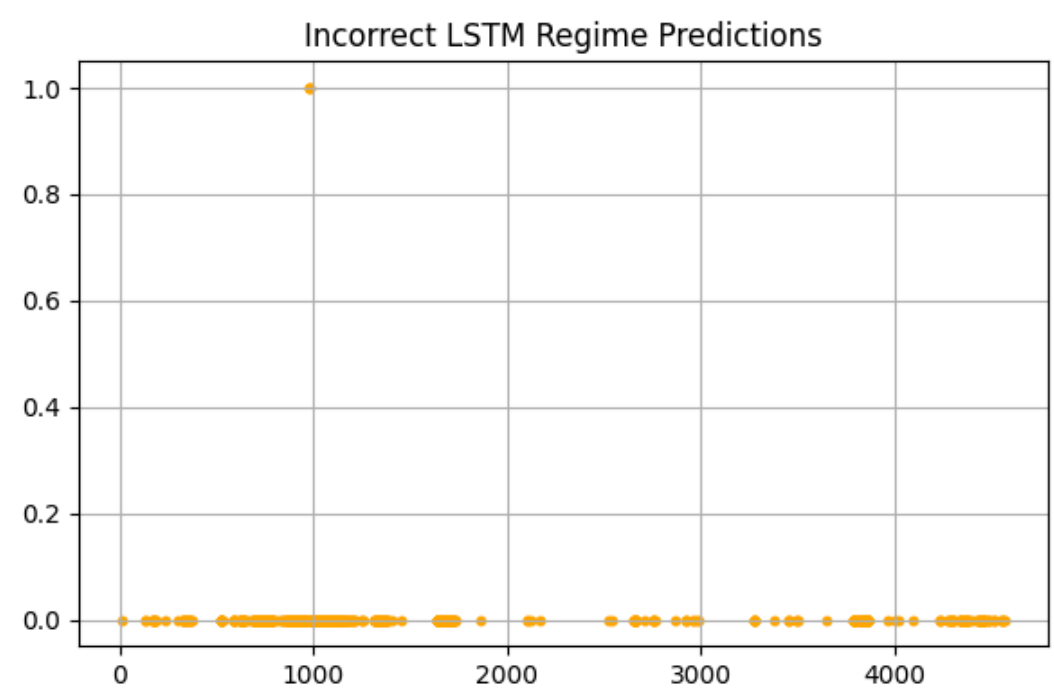
```
plt.figure(figsize=(16, 5))
plt.plot(y, label='True Regime', color='black', alpha=0.5, linewidth=1)
plt.plot(predicted_classes, label='Predicted Regime (LSTM)', color='red', linewidth=1)
plt.title("LSTM-Predicted vs Actual Market Regimes")
plt.xlabel("Time Steps")
plt.ylabel("Regime (0 = Bear, 1 = Bull)")
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.tight_layout()
plt.show()
```



Black line: Actual market regime Red line: What your LSTM model predicted

```
mismatch = (y != predicted_classes)
plt.figure(figsize=(6, 4))
plt.scatter(np.where(mismatch), y[mismatch], color='orange', label='Mismatches', s=10)
plt.title("Incorrect LSTM Regime Predictions")
plt.grid(True)
plt.tight_layout()
plt.show()
```



Every orange dot = LSTM predicted the wrong regime on that day